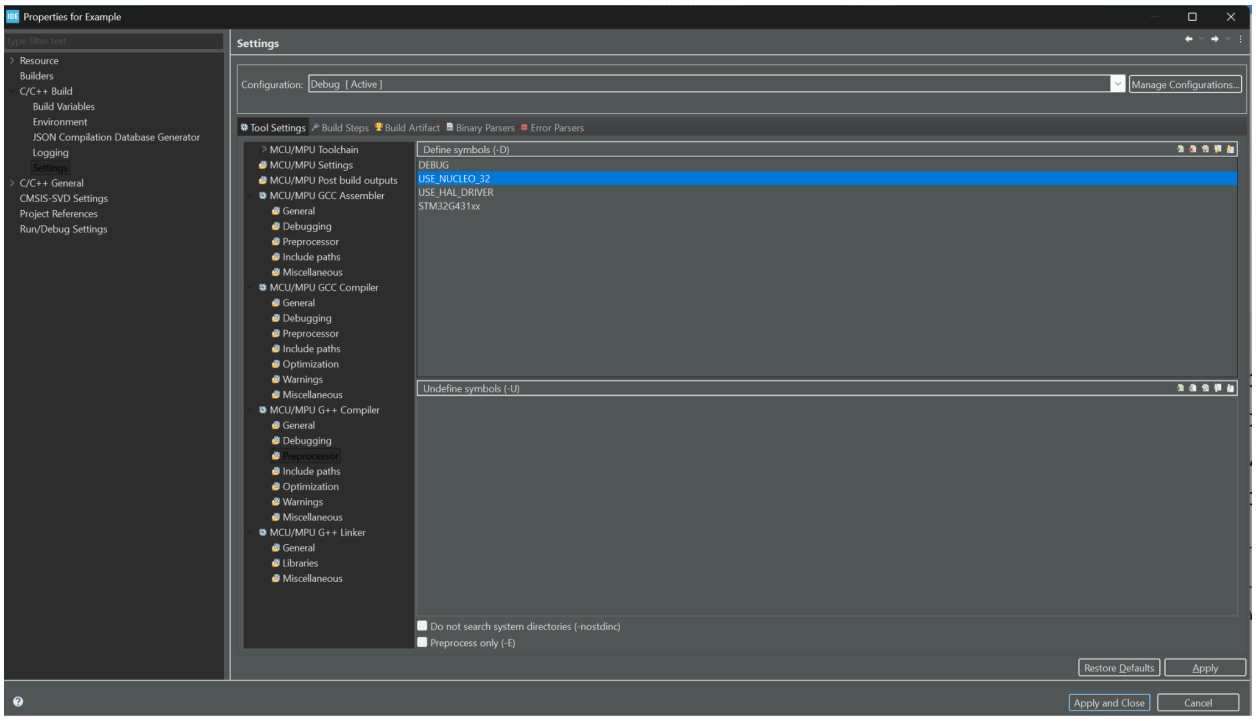
This folder contains a ready to use basic setup for our STM32 that can loaded into CubeIDE. To do this open CubeIDE and select File → Open Projects from File System and select the Example folder.

These options should automatically be set but if not they must be changed:

1.

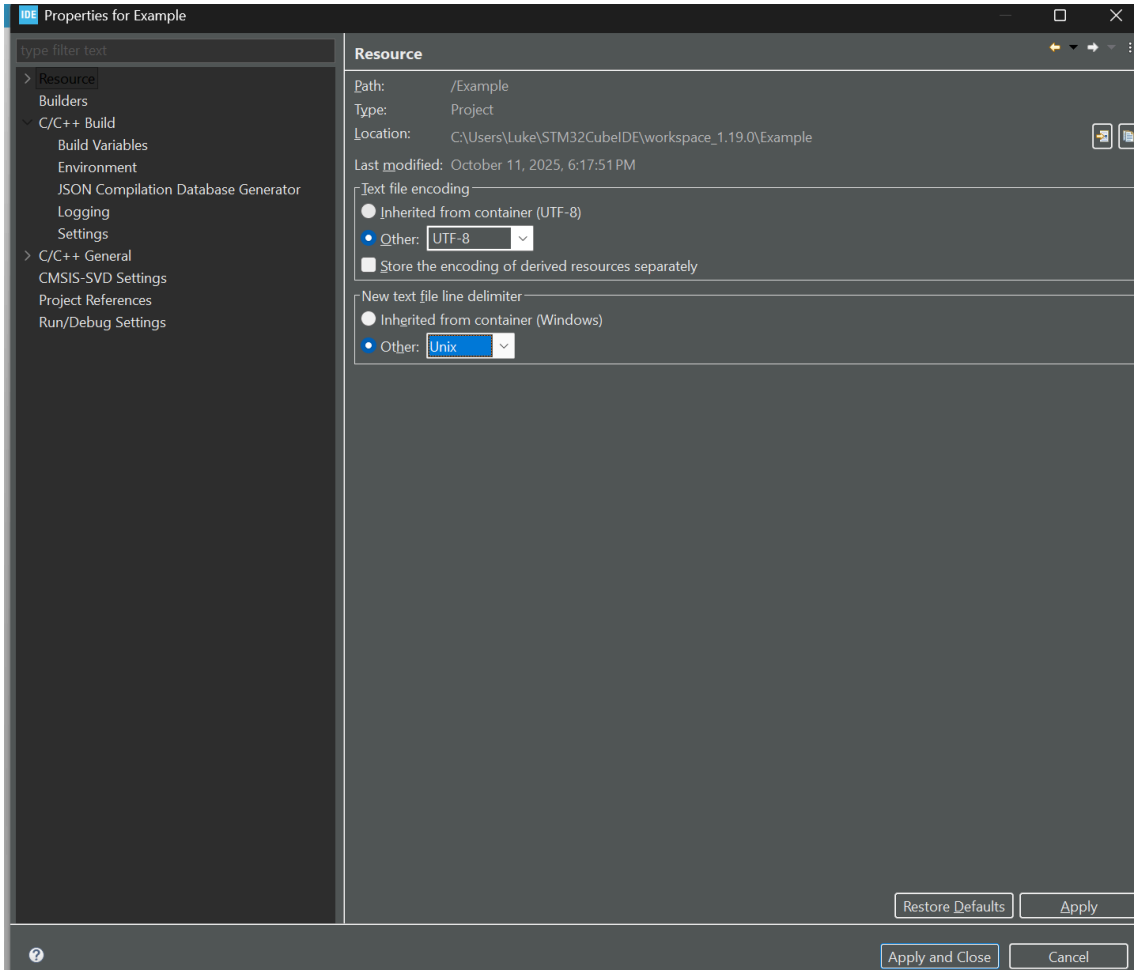
Project --> Properties --> C/C++ Build --> Settings --> G++ Compiler --> Preprocessor → Change USE_NUCLEO_64 to USE_NUCLEO_32



Do this for GCC compiler as well (g++ is for c++)
Change the configuration to Release and repeat these steps

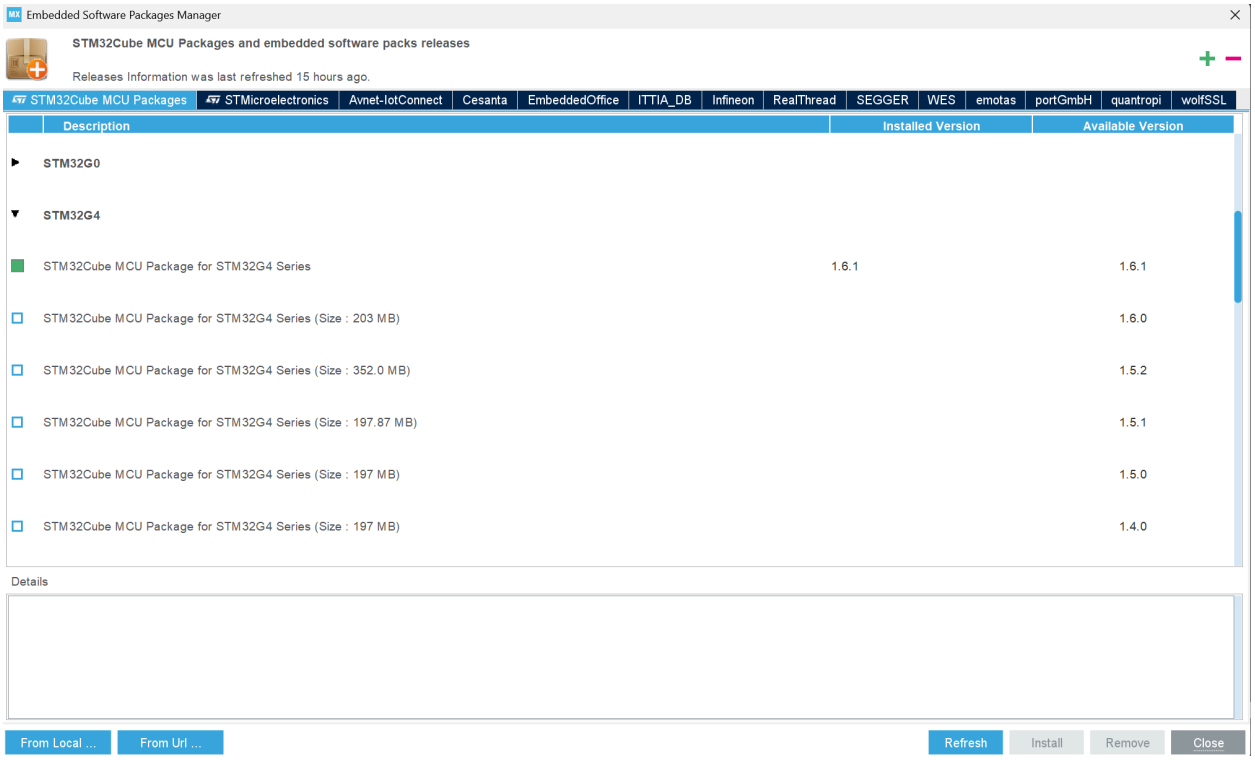
2.

Project --> Properties --> Resource → Set text file encoding and line endings



This just gets rid of the warning sign on the project (if it appears). Other warnings can be viewed in the Problems tab.

3. Download MCU package: Help → Configuration tool → Manage Embedded Software Packages → Scroll to STM32G4 and download the latest version. You have to sign in to do this unfortunately.

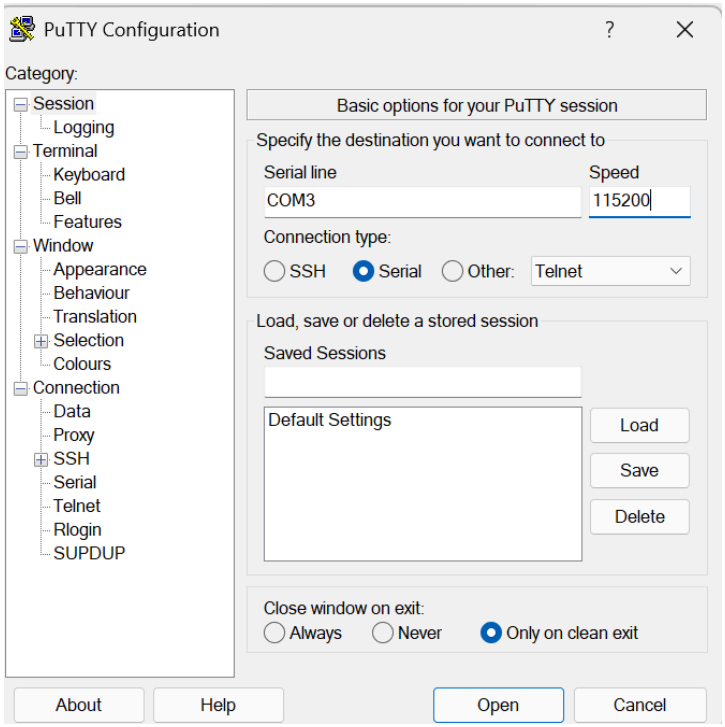


Coding:

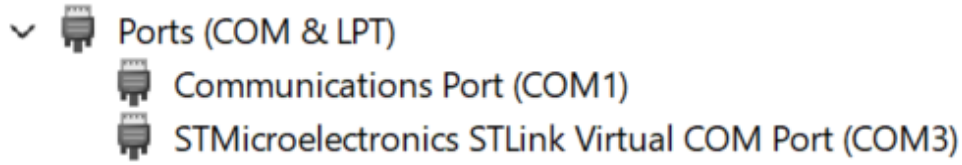
1. There is a main_cpp.cpp file in the Src folder that uses a basic setup loop program. This can be safely edited. It is called in main.c. Editing any of the initial files (e.g. main.c) should be done carefully - any code outside of the user code blocks can be removed if the code needs to be regenerated. In the actual code we will not have any connection to the .c files except through main_cpp. Note the use of extern “C” when declaring main_cpp, this is needed for the C compiler to use it. This works the other way, but CubeIDE automatically adds this to its C files, guarded by #ifdef __cplusplus.
2. The .ioc file contains all MCU settings. If edited, new code should be generated by doing Project → Generate Code. Project → Generate Report will create a .txt snapshot of current settings.
3. The BSP (board supported packages) automatically sets up and allows for some abstraction for 2 things: controlling the on-board LED and printing to the virtual COM port. You can see these are initialized in main.c:

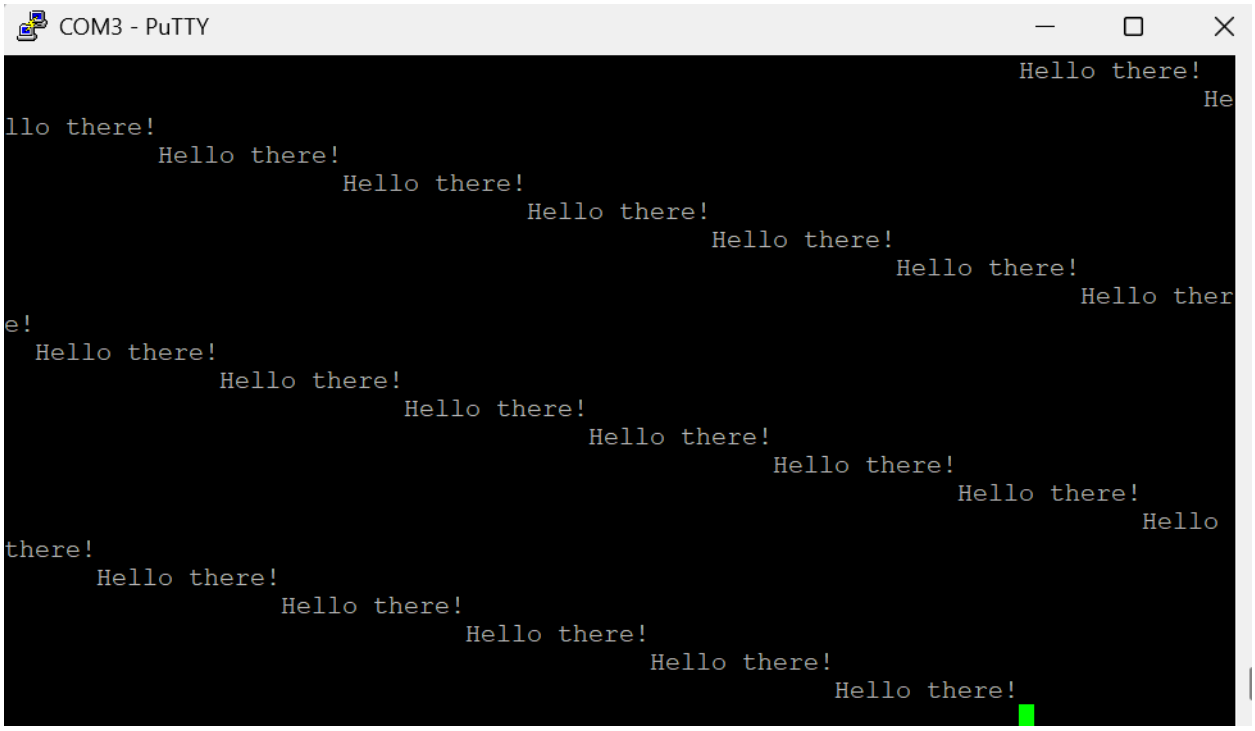
```
/* Initialize leds */
BSP_LED_Init(LED_GREEN);

/* Initialize COM1 port (115200, 8 bits (7-bit data + 1 stop bit), no parity */
BspCOMInit.BaudRate   = 115200;
BspCOMInit.WordLength = COM_WORDLENGTH_8B;
BspCOMInit.StopBits   = COM_STOPBITS_1;
BspCOMInit.Parity     = COM_PARITY_NONE;
BspCOMInit.HwFlowCtl  = COM_HWCONTROL_NONE;
```

This feature enables using printf() to print to the VCP. To view the output, use something like PuTTy:

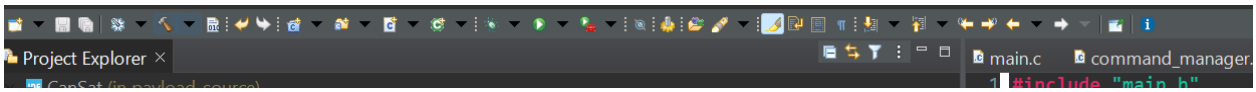


Set COM# not to match port in device list:





4. Press the hammer icon to build code (choose Debug), any issues are printed in the console. Then press green arrow button to upload.



5. Source files can be placed anywhere, keeping them in Src folder is fine. Header files must be placed in Inc/ otherwise they need to be manually added to the linker.
6. All pin initializations and settings can be viewed in the main.c file. It's a good idea to look over this file and understand it.